Übung zur PLF: Ein Autoverleih

> Klasse: 5AKIF
> Training für die PLF am MI, 4. November 2026
> Arbeitszeit in der PLF: 3 UE

Hinweise zu dieser Übung

Diese Übung zeigt Ihnen, wie die Angabe und das Testprogramm in der PLF aussehen. Die PLF hat den gleichen Aufbau: ein Klassenmodell, eine Beschreibung der Klassen und ein Testprogramm, das Punkte vergibt.

Laden Sie die Datei *CarRentalService.7z* aus den Übungen zum Kapitel *Types* herunter. Entpacken Sie die Datei und öffnen Sie die Datei *CarRentalService.sln*.

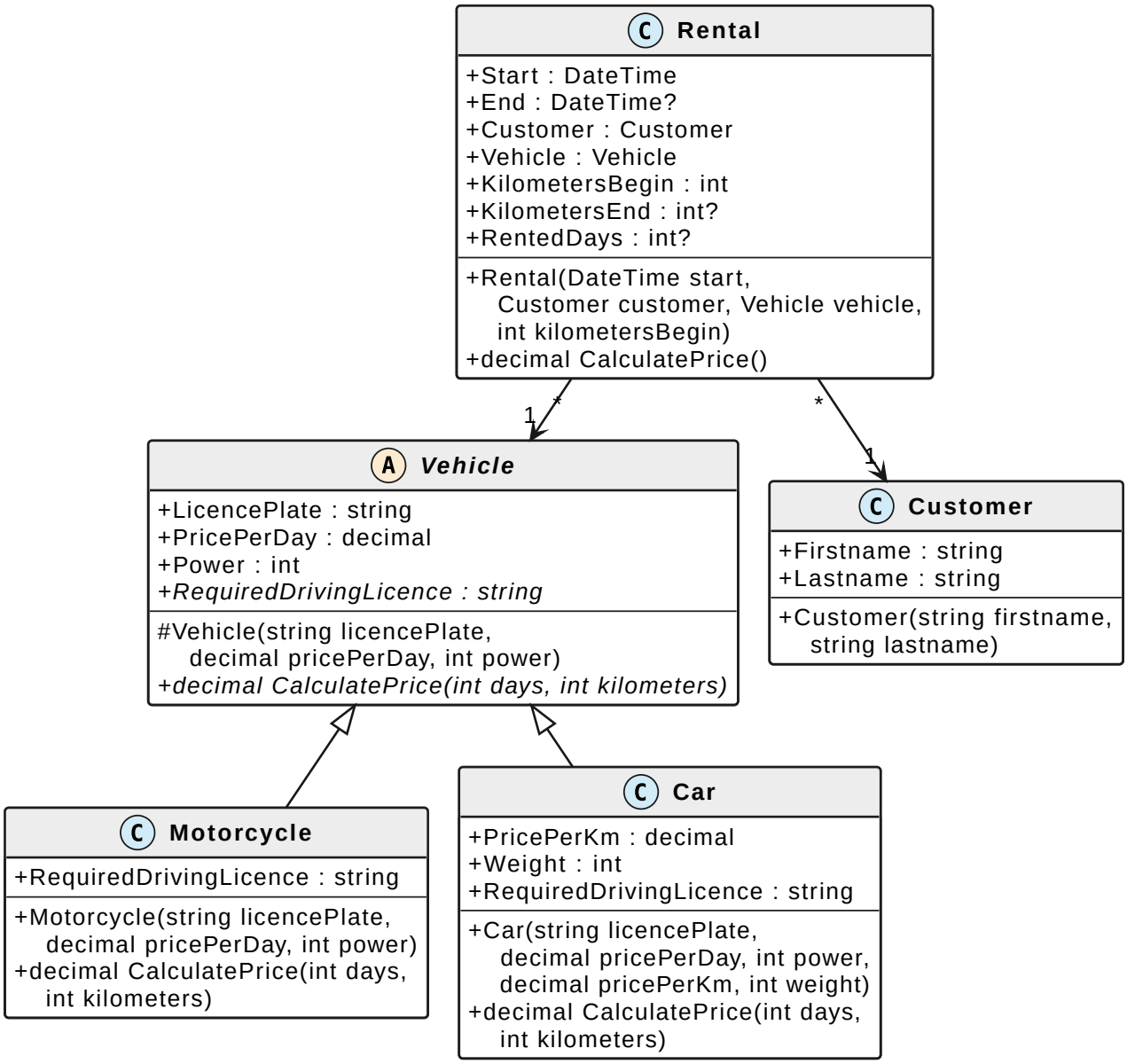


In der PLF finden Sie die Solution auf dem Laufwerk *Z*. Dort füllen Sie zuerst die Datei *README.md* mit Ihrem Namen aus. In dieser Übung ist die *README.md* dagegen die Angabe, Sie müssen nichts eintragen.

Aufgabe: Ein Autoverleih

Ein Autoverleih vermietet Motorräder und Autos. Bei Motorrädern zahlen Sie einen Preis pro Tag. Bei Autos zahlen Sie zusätzlich für jeden gefahrenen Kilometer. Für jede Miete speichert das System einen Mietvorgang (*Rental*). Ein Mietvorgang ist offen, bis das Fahrzeug zurückgegeben wird.

Klassenmodell



Klassenbeschreibung

Private Felder, die Sie für die Implementierung brauchen, legen Sie selbst an.

Klasse Vehicle

Die abstrakte Basisklasse für Autos und Motorräder. Alle Properties sind read-only.

- **LicencePlate** ist das Kennzeichen (die Nummerntafel) des Fahrzeuges.
- **PricePerDay** ist der Mietpreis für jeden begonnenen Tag.
- **Power** ist die Leistung des Fahrzeuges in kW.
- Der Konstruktor ist *protected*.
- **RequiredDrivingLicence** ist ein abstraktes Property. Es liefert die Führerscheinklasse, die man für das Fahrzeug braucht.

- **CalculatePrice()** ist eine abstrakte Methode. Sie berechnet den Preis aus den Miettagen (*days*) und den gefahrenen Kilometern (*kilometers*).

Klasse Motorcycle

Alle Properties sind read-only.

- **RequiredDrivingLicence** liefert immer *"A"*.
- **CalculatePrice()** berechnet den Preis mit der Formel *days* × *PricePerDay*. Die Kilometer spielen keine Rolle.

Klasse Car

Alle Properties sind read-only.

- **PricePerKm** ist der Preis pro gefahrenem Kilometer.
- **Weight** ist das Gewicht in kg.
- **RequiredDrivingLicence** liefert *"B"*, wenn das Auto weniger als 3500 kg wiegt. Sonst liefert es *"C"* (LKW).
- **CalculatePrice()** berechnet den Preis mit der Formel *days* × *PricePerDay* + *kilometers* × *PricePerKm*.

Klasse Customer

Die Klasse speichert nur den Vor- und Nachnamen des Kunden. Der Konstruktor setzt alle Properties. Alle Properties sind read-only.

Klasse Rental

Die Klasse speichert einen Mietvorgang.

- **Start**, **Customer**, **Vehicle** und **KilometersBegin** werden im Konstruktor gesetzt und sind read-only.
- **Start** ist der Beginn der Miete.
- **KilometersBegin** ist der Kilometerstand am Beginn der Miete.
- **End** ist das Ende der Miete. Es ist *null*, solange das Fahrzeug nicht zurückgegeben wurde. Bei der Rückgabe wird es von außen gesetzt.
- **KilometersEnd** ist der Kilometerstand am Ende der Miete. Es ist *null*, solange das Fahrzeug nicht zurückgegeben wurde. Bei der Rückgabe wird es von außen gesetzt.
- **RentedDays** ist die Anzahl der begonnenen Tage und read-only. Ist *End null*, ist auch *RentedDays null*. Eine Miete, die am selben Tag endet, dauert 1 Tag.
- **CalculatePrice()** berechnet den Preis der Miete:
  - Ist *End null*, liefert die Methode 0.
  - Ist *KilometersEnd null*, liefert die Methode 0.
  - Sonst liefert sie das Ergebnis von *CalculatePrice()* des gemieteten Fahrzeuges. Übergeben

Sie dabei *RentedDays* und die gefahrenen Kilometer (*KilometersEnd* – *KilometersBegin*).

Beispiel: Ein Auto mit *PricePerDay* 10 Euro und *PricePerKm* 5 Euro wird vom 10.11.2021 bis zum 15.11.2021 gemietet. Der Kilometerstand ist am Beginn 100 km und am Ende 200 km.

- *RentedDays* ist 6 (10., 11., 12., 13., 14. und 15. November).
- *CalculatePrice()* liefert 6 × 10 + 100 × 5 = 560 Euro.



- Ziehen Sie 2 *DateTime* Werte voneinander ab, bekommen Sie einen *TimeSpan*. Die begonnenen Tage bekommen Sie mit diesem Ausdruck:

```
(int)(end - start).TotalDays + 1
```

- Das funktioniert nur mit *DateTime*, nicht mit *DateTime?*. Prüfen Sie daher zuerst, ob *End null* ist. Danach bekommen Sie den Wert mit `End.Value`. Das gilt genauso für *KilometersEnd*.
- Ein *decimal* Literal schreiben Sie mit *M* am Ende, z. B. `0.5M`.

Durchzuführende Arbeiten

Legen Sie jede Klasse in einer eigenen Datei an. Die Dateien *Program.cs* und *TestHelpers.cs* dürfen Sie **nicht** ändern.

Program.cs ist ein Testprogramm. Es kompiliert erst, wenn alle Klassen mit allen Properties und Methoden aus dem Modell existieren. Achten Sie auch auf die Namen der Konstruktorparameter, denn das Testprogramm verwendet sie (z. B. `pricePerDay: 10`). Legen Sie daher zuerst alle Klassen an. Methoden und Properties, die Sie noch nicht implementiert haben, werfen eine *NotImplementedException*. Danach starten Sie das Programm im Projekt *CarRentalService.Application*. Es zeigt bei jeder Prüfung *OK* oder *FAILED*. Wirft eine Prüfung eine Exception, gilt sie als nicht erfüllt. Eine Exception außerhalb einer Prüfung (z. B. beim Anlegen der Testdaten) beendet das Programm.

Eine vollständige Lösung zeigt bei allen 43 Prüfungen *OK*. Am Ende zeigt das Programm die erreichten Punkte und die Note:

```
Result of the program in C:\...\CarRentalService.Application:
43 of 43 checks passed.
48 of 48 points. Grade: Sehr gut.
```

Bewertung

Um eine positive Beurteilung erreichen zu können, muss das Programm kompilieren. Programme, die nicht kompilieren oder abstürzen, werden mit *Nicht genügend* bewertet. Die Tabelle zeigt alle Prüfungen des Testprogramms. Die Texte sind die Texte der Bildschirmausgabe. Jede Prüfung mit *OK* bringt die Punkte in der Spalte *P*.

Nr	Prüfung (48 Punkte in Summe)	P
Checking class implementation		
1	Rental does not have a default constructor.	1
2	Rental.Start is read-only.	1
3	Rental.Customer is read-only.	1
4	Rental.Vehicle is read-only.	1
5	Rental.KilometersBegin is read-only.	1
6	Rental.RentedDays is read-only.	1
7	Rental.End, KilometersEnd and RentedDays are nullable.	1
8	Customer does not have a default constructor.	1
9	Customer is immutable.	1
10	Vehicle does not have a default constructor.	1
11	Vehicle is abstract.	1
12	Vehicle has a protected constructor.	1
13	Vehicle is immutable.	1
14	Vehicle.PricePerDay is decimal and Vehicle.Power is int.	1
15	Vehicle.RequiredDrivingLicence is abstract.	1
16	Vehicle.CalculatePrice() is abstract.	1
17	Motorcycle does not have a default constructor.	1
18	Motorcycle is immutable.	1
19	Car does not have a default constructor.	1
20	Car is immutable.	1
21	Car.PricePerKm is decimal and Car.Weight is int.	1
Checking constructors		
22	Constructor of Customer sets the values.	1
23	Constructor of Motorcycle sets the values.	1
24	Constructor of Car sets the values.	1
25	Constructor of Rental sets the values.	1
Checking Motorcycle		
26	The required driving licence for motorcycles is A.	1
27	Motorcycle.CalculatePrice() returns days x PricePerDay.	2
Checking Car		
28	The required driving licence for cars < 3500 kg is B.	1
29	Car.CalculatePrice() returns days x PricePerDay + km x PricePerKm.	2
30	The required driving licence for cars >= 3500 kg is C.	1
Checking Rental		
31	The default value of Rental.KilometersEnd is null.	1
32	The default value of Rental.End is null.	1
33	Without an end date Rental.RentedDays is null.	1

Nr	Prüfung (48 Punkte in Summe)	P
34	Without End and KilometersEnd Rental.CalculatePrice() returns 0.	1
35	Rental.RentedDays returns the number of days started.	2
36	Without KilometersEnd Rental.CalculatePrice() returns 0.	1
37	Rental.CalculatePrice() uses the driven kilometers.	2
38	Without End Rental.CalculatePrice() returns 0.	1
39	Rental.RentedDays is 1 if the vehicle is returned on the same day.	1
40	Rental.RentedDays works across the end of a month.	1
41	Rental.RentedDays works across the end of a year.	1
42	Rental.CalculatePrice() charges the days if no kilometers were driven.	1
43	Rental.CalculatePrice() uses the price calculation of the rented vehicle.	2

48 - 42 Punkte: Sehr gut, 41 - 36 Punkte: Gut, 35 - 30 Punkte: Befriedigend, 29 - 24 Punkte: Genügend, 23 - 0 Punkte: Nicht genügend.